

# Extra assignment: circle/sphere fitting with known radius

## Iterative Method: Geometric (Radial) Least Squares

### Theory:

#### Iterative Method : Radial Least Squares

Given  $N$  points  $\{p_1, p_2, \dots, p_n\}$  where  $p_i = [x_i, y_i]^T$  and a known radius  $R$  we find the center  $c = [x_0, y_0]^T$  that best fits the circle.

For each point  $p_i$ , the distance to the center is:

$$d_i = \|p_i - c\| = \sqrt{(x_i - x_0)^2 + (y_i - y_0)^2}$$

The radial error (residual) is:  $r_i = d_i - R$

Radial least squares minimizes the sum of squared radial errors:

$$J(x_0, y_0) = \sum_{i=1}^N r_i^2 = \sum_{i=1}^N (\sqrt{(x_i - x_0)^2 + (y_i - y_0)^2} - R)^2$$

This is a non-linear optimization problem because of the squared root term. Therefore an iterative method is required.

→ I use the Levenberg-Marquardt algorithm, which is the following:

1. Start with an initial guess  $c^{(0)}$  (e.g. centroid of points)

2. Compute residuals:  $r_i = \|p_i - c^{(k)}\| - R$  (to know how "wrong" our current guess is)

3. Compute the Jacobian matrix  $J$  (compute how each error changes when we move the center)

4. Update:  $c^{(k+1)} = c^{(k)} - (J^T J + \lambda I)^{-1} J^T r$  (move center in the direction that reduces error the most)

5. Repeat until  $\|c^{(k+1)} - c^{(k)}\| < \epsilon$

\*  $J$  is calculated as: For each point  $i$ , the partial derivatives are:

$$\frac{\partial r_i}{\partial x_0} = \frac{x_0 - x_i}{d_i}; \quad \frac{\partial r_i}{\partial y_0} = \frac{y_0 - y_i}{d_i} \quad \rightarrow \quad J = \begin{bmatrix} (x_0 - x_1)/d_1 & (y_0 - y_1)/d_1 \\ (x_0 - x_2)/d_2 & (y_0 - y_2)/d_2 \\ \vdots & \vdots \\ (x_0 - x_n)/d_n & (y_0 - y_n)/d_n \end{bmatrix}$$

For sphere fitting:  $J(x_0, y_0, z_0) = \sum_{i=1}^N (\sqrt{(x_i - x_0)^2 + (y_i - y_0)^2 + (z_i - z_0)^2} - R)^2$

Same iterative approach, but we optimize 3 parameters instead of 2.

## Implementation:

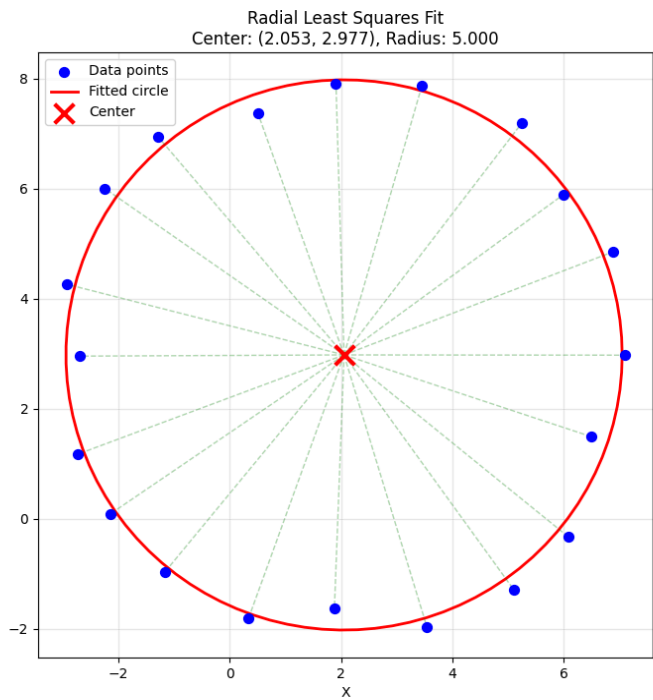
### ***Example 1: Noisy circle data***

Converged in 4 iterations

*True center:* (2.000, 3.000)

*Fitted center:* (2.053, 2.977)

*RMS error:* 0.208705

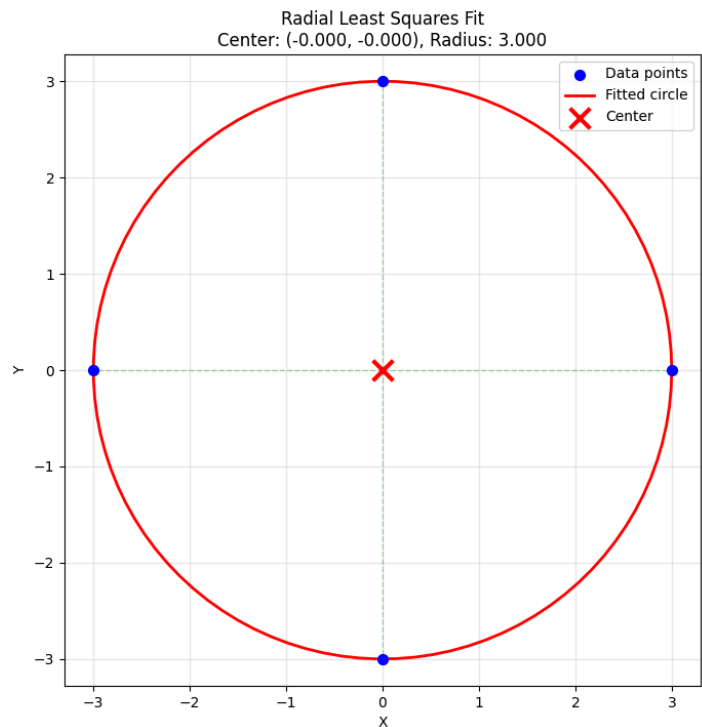


### ***Example 2: Simple 4-point case***

Converged in 4 iterations

*Fitted center:* (-0.000000, -0.000000)

*Expected:* (0.0, 0.0)



## **Non-Iterative Method: Algebraic Least Squares**

### **Theory:**



## Non-Iterative Method: Algebraic Least Squares

Given  $N$  points  $\{p_1, p_2, \dots, p_n\}$  where  $p_i = [x_i, y_i]^T$  and a known radius  $R$ , find the center  $C = [x_0, y_0]^T$  that best fits the circle. This method works for the over-determined case where  $N \geq 3$ .

We know that:  $(x_i - x_0)^2 + (y_i - y_0)^2 = R^2$

For each point, define the algebraic error:  $e_i = (x_i - x_0)^2 + (y_i - y_0)^2 - R^2$

We want to minimize the sum of squared algebraic errors:

$$J(x_0, y_0) = \sum_{i=1}^N e_i^2 = \sum_{i=1}^N [(x_i - x_0)^2 + (y_i - y_0)^2 - R^2]^2$$

This is a least squares optimization problem.

Expanding the circle equation:  $x_i^2 - 2x_0x_i + y_i^2 - 2y_0y_i + x_0^2 + y_0^2 - R^2 = 0$

We introduce auxiliary variable:  $k = x_0^2 + y_0^2 - R^2$

This transforms the equation to:  $-2x_0x_i - 2y_0y_i + k = -x_i^2 - y_i^2$

Now the unknowns in the equation appear linearly:  $z = [k, x_0, y_0]^T$

→ Reformulation as Linear Least Squares: For  $N$  points, the algebraic error for each point becomes:  $e_i = (1)k + (-2x_i)x_0 + (-2y_i)y_0 - (-x_i^2 - y_i^2)$

We want to minimize:  $J(z) = \|Az - b\|^2 = \sum_{i=1}^N (A_{i2}z - b_i)^2$

$$\text{where: } A = \begin{bmatrix} 1 & -2x_1 & -2y_1 \\ 1 & -2x_2 & -2y_2 \\ \vdots & \vdots & \vdots \\ 1 & -2x_n & -2y_n \end{bmatrix} \quad b = \begin{bmatrix} -x_1^2 - y_1^2 \\ -x_2^2 - y_2^2 \\ \vdots \\ -x_n^2 - y_n^2 \end{bmatrix}$$

This is a linear least squares optimization problem in  $z$ .

To find the minimum, we take the derivative and set to zero:

$$\frac{\partial J}{\partial z} = 2A^T A z - 2A^T b = 0 \rightarrow \text{solving for } z: A^T A z = A^T b$$

$$\rightarrow z^* = (A^T A)^{-1} A^T b \quad \text{to extract}$$

$$\text{cent: } x_0 = z^*[1], \quad y_0 = z^*[2]$$

+ Use same optimization approach for the sphere case.

## Implementation:

### **Example 1: Minimal case (N=3, exactly determined)**

Objective  $J(z^*)$  minimized to: 2.524355e-28

$k$  from solution: -25.000000

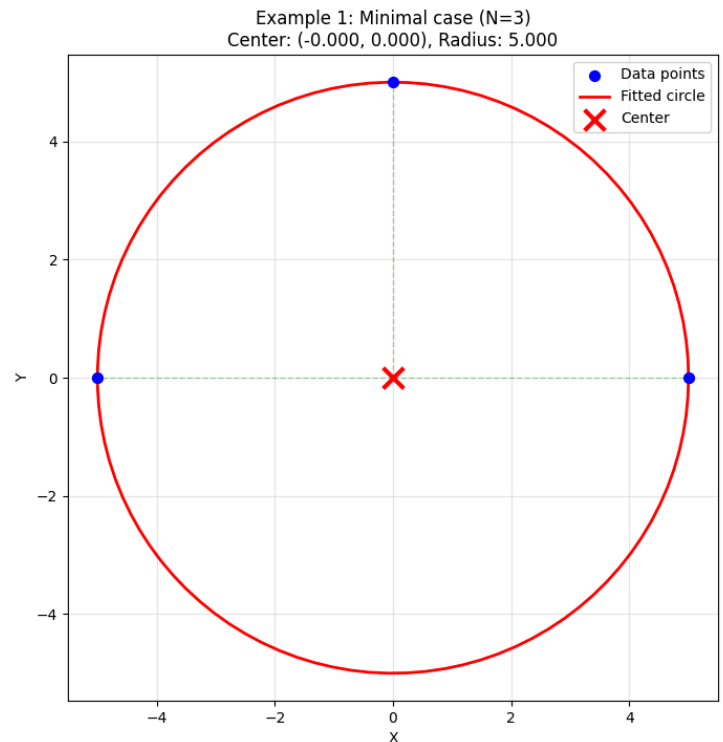
$k$  expected ( $x_0^2 + y_0^2 - R^2$ ): -25.000000

Difference: 1.065814e-14

Fitted center: (-0.000000, 0.000000)

Expected center: (0.0, 0.0)

RMS error: 0.0000000000



### **Example 2: Large over-determined case (N=100)**

Objective  $J(z^*)$  minimized to: 1.600996e+02

Number of equations (N): 100

Number of unknowns: 3 ( $k$ ,  $x_0$ ,  $y_0$ )

$k$  from solution: -40.523703

$k$  expected ( $x_0^2 + y_0^2 - R^2$ ): -40.468760

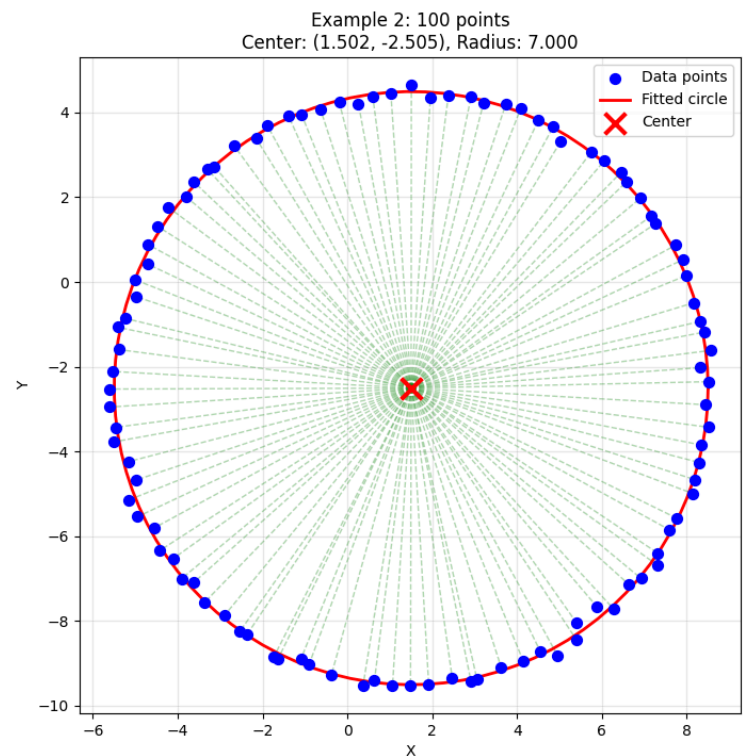
Difference: 5.494268e-02

True center: (1.500, -2.500)

Fitted center: (1.502, -2.505)

Error: 0.005384

RMS residual: 0.09061



## **Bonus:**

I looked up other methods used in different applications.

One iterative method that used for detecting outliers is Ransac. It is an estimation method that can handle datasets with a significant proportion of outliers (up to 50%).

Unlike least squares methods that use all data points, RANSAC:

1. Randomly samples minimal subsets of data
2. Fits a model to each subset
3. Counts how many points agree with the model (inliers)
4. Selects the model with the most inliers

The inliers represent points that fit the model within a distance threshold. The algorithm of Ransac that we can use to fit the circle is the following:

### **Algorithm**

**Input:** N points, known radius R, threshold  $\tau$ , iterations K

### **Steps:**

For  $k = 1$  to K iterations:

1. SAMPLE: Randomly select 2 points
2. FIT: Compute candidate centers (intersection of two circles of radius R)
3. EVALUATE: Count inliers where  $|\text{distance}(p_i, c) - R| < \tau$
4. UPDATE: Keep model with most inliers

REFINE: Apply Radial LS on final inlier set for optimal accuracy

**Output:** Refined center from inliers

### **Note:**

To compute the center from two points, knowing the distance between points:  $d = \|p_2 - p_1\|$

- If  $d > 2R$ : No solution (skip sample)
- If  $d = 2R$ : One center (midpoint)
- If  $d < 2R$ : Two centers (circle intersections)

For  $d < 2R$ , the two candidate centers are:  $c_{1,2} = (p_1 + p_2)/2 \pm h \cdot v$

where  $h = \sqrt{(R^2 - (d/2)^2)}$  and  $v$  is perpendicular to line  $p_1p_2$ .

## Implementation:

### **Example1: 30 percent outliers**

RANSAC: 29 points,  $R=5.0$ ,  $T=0.4$ , 1000 iterations

Best model: 22/29 inliers (75.9%)

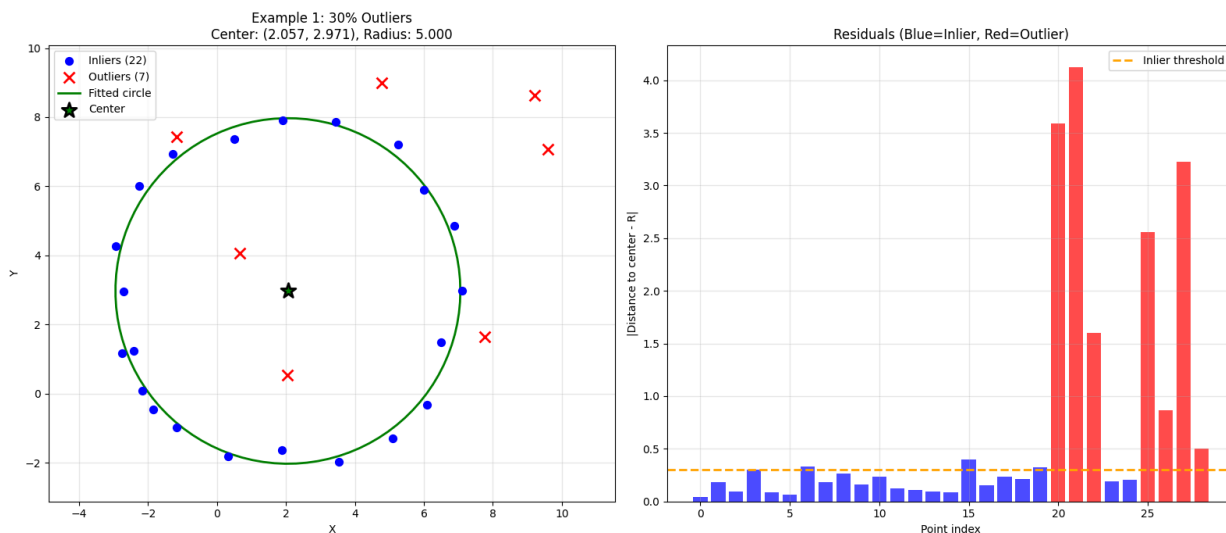
Initial center: (1.895917, 2.980437)

Refined center: (2.057165, 2.971327)

True center: (2.000, 3.000)

Fitted center: (2.057, 2.971)

Error: 0.063953



### **Example2: 70 percent outliers (This is challenging! RANSAC needs many iterations)**

RANSAC: 40 points,  $R=4.5$ ,  $T=0.4$ , 5000 iterations

Best model: 17/40 inliers (42.5%)

Initial center: (2.662094, 2.631968)

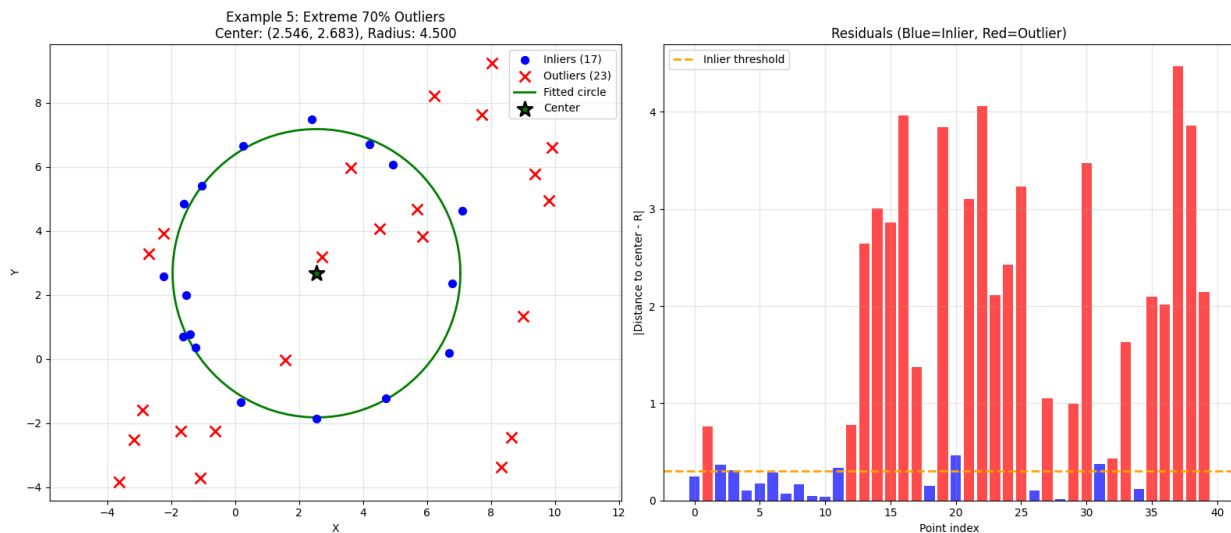
*Refined center:* (2.546450, 2.682940)

*True center:* (2.500, 2.500)

*Fitted center:* (2.546, 2.683)

*Error:* 0.188745

✓ RANSAC succeeded despite 70% outliers!



Another non-iterative method I found in a lot of applications is Hough Transform. Unlike optimization-based approaches (algebraic LS, geometric LS), Hough Transform uses a discrete voting scheme.

The main idea is that if a point  $p_i$  lies on a circle with radius  $R$  and center  $c = (x_0, y_0)$ , then:

$\|p_i - c\| = R$ ; This means that the center  $c$  must lie on a circle of radius  $R$  centered at point  $p_i$ .

When we have multiple points on the same circle, each point  $p_i$  votes for a circle of possible centers, and the true center receives votes from all points, so the location with maximum votes is the fitted center.

The algorithm goes like this:

**Input:**  $N$  points  $\{(x_i, y_i)\}$ , known radius  $R$

**Steps:**

**1. Initialize Accumulator:**

- Create 2D accumulator array  $H[x, y]$  covering the search space
- Set resolution (grid spacing):  $\delta x, \delta y$
- Initialize all votes to zero:  $H[x, y] = 0$

**2. Voting Process:**

For each point  $p_i = (x_i, y_i)$ :

For each angle  $\theta \in [0, 2\pi]$ :

Compute candidate center:

$$x_0 = x_i + R \cdot \cos(\theta)$$

$$y_0 = y_i + R \cdot \sin(\theta)$$

Increment vote in accumulator:

$$H[\text{round}(x_0 / \delta x), \text{round}(y_0 / \delta y)] += 1$$

**3. Find Peak:**

- Locate maximum in accumulator:  $(x_0^*, y_0^*) = \text{argmax } H[x, y]$
- This is the fitted center

**Output:** Center  $(x_0^*, y_0^*)$  with maximum votes

**Implementation:**

***Example: Circle with 20 points (moderate noise)***

*Accumulator size: (123, 127)*

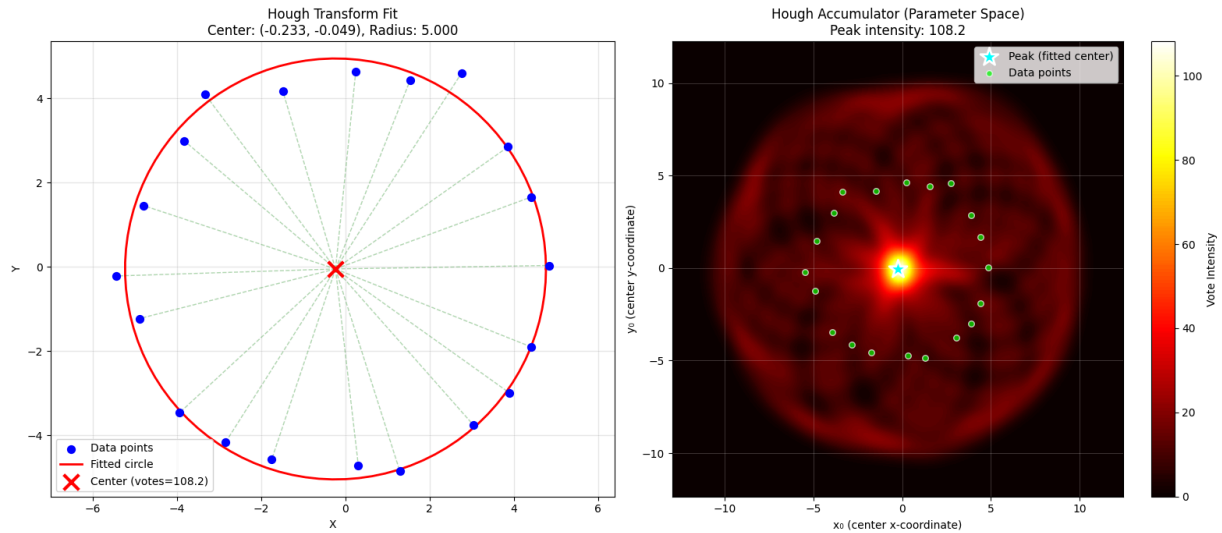
*True center: (0.000, 0.000)*

*Fitted center: (-0.233, -0.049)*

*Error: 0.238142*

*RMS error: 0.243233*





## References:

<https://link.springer.com/article/10.1007/BF00939613>

<https://www.wasyresearch.com/mathematical-geometrical-fitting-direct-least-square-fitting-of-circle-geometry-with-tutorial-and-code/>

<https://www.wasyresearch.com/mathematical-geometrical-fitting-non-linear-geometry-least-squared-fitting-with-tutorial/>

<https://imagej.net/plugins/hough-circle-transform>

<https://medium.com/@saurabh.dasgupta1/ransac-algorithm-for-circles-1e10a5d8617>

## Note:

I used AI to generate the code examples.